

IAWAKEE – Lach und Sachgeschichten aus Mikroklimaforschung und Informatik



Inhaltsübersicht:

Hintergrund

Einführung Softwaredesign

Einführung Coding-Paradigmen

- Imperative Programmierung

- Objekt-Orientierte-Programmierung

- Deklarative Programmierung

OOP Pattern

Zusammenfassung

- Steigende Bedeutung der Mikroklimaforschung
 - ▶ Große Datensammlungen
- Steigende Bedeutung der Informatik
- KI-Hype
 - ▶ Soziale / Ökologische Kosten

- Steigende Bedeutung der Mikroklimaforschung
 - ▶ Große Datensammlungen
- Steigende Bedeutung der Informatik
 - KI-Hype
 - ▶ Soziale / Ökologische Kosten

- Steigende Bedeutung der Mikroklimaforschung
 - Rückzugsort für bedrohte Arten
 - Resilienz in Landwirtschaft
- Herausforderungen:
 - Klimaforschung = Statistik
 - > Klimadaten (selbst für Mikroklimaforschung) umfassen riesige Gebiete (ganze Städte, kleinere Wälder, ganz Brandenburg)
 - > Also: Computer
 - Außerdem **große Zeiträume**

Siehe nächste Notiz ⇒⇒

- Steigende Bedeutung der Mikroklimaforschung
 - Große Datensammlungen
- Steigende Bedeutung der Informatik
 - KI-Hype
 - Soziale / Ökologische Kosten

- Steigende Bedeutung der Informatik
 - etwas selbsterklärend
 - Digitalisierung
 - Bürokratiebewältigung
- KI-Hype
 - Medienwirksam
 - Hohe *soziale* + *ökologische* Kosten

- Aufgabe zur Entwicklung einer Optimierungssoftware für Klima-Maßnahmen
- Verlässliche, hochwertige Software erforderlich

⇒ Softwaredesign von zentraler Bedeutung

- Aufgabe zur Entwicklung einer Optimierungssoftware für Klima-Maßnahmen
- Verlässliche, hochwertige Software erforderlich

⇒ Softwaredesign von zentraler Bedeutung

- Aufgabe zur Entwicklung einer Optimierungssoftware für Klima-Maßnahmen
 - Klima-Maßnahmen
 - > Werden auf Flächen angewandt
 - > Flächen müssen gekauft werden
 - > Bsp: Bestandsbegrünung, Anlegen von Baumgruppen
 - > Werden optimiert
- Verlässliche, hochwertige Software erforderlich
 - Unfassbare Geldbeträge (hunderte Millionen €) laufen über software
 - > Kleine Fehler verursachen Schäden in Millionenhöhe
 - Verwendung in der Regierung
 - Auch von Menschen ohne Computeraffinität

Software-Design	Das Planen von Software um bestimmte Ziele zu erfüllen
Design Patterns ¹	Standard Lösungen für Probleme bei der Softwareentwicklung Werden entdeckt, <u>nicht</u> erfunden
Paradigmen	Fundamentale Programmierstile

¹ „Entwurfsmuster“

IAWAKEE – Lach und Sachgeschichten aus Mikroklimaforchung und Informatik

Einführung Softwaredesign

Softwaredesign – Einführung

Software-Design	Das Planen von Software um bestimmte Ziele zu erfüllen
Design Patterns ¹	Standard Lösungen für Probleme bei der Softwareentwicklung Werden entdeckt, <u>nicht</u> erfunden
Paradigmen	Fundamentale Programmierstile

¹ „Entwurfsmuster“

Software-Design Das Planen von Software um bestimmte Ziele zu erfüllen

Design Patterns² Standard Lösungen für Probleme bei der Softwareentwicklung
Patterns werden entdeckt, nicht erfunden

Paradigmen Fundamentale Programmierstile

Welche Ziele soll die Software nun erfüllen?

- Testbarkeit
 - ▶ Automatisierbarkeit
- Wartbarkeit
- Erweiterbarkeit
- Kompatibilität

⇒ Standards setzen / Entscheidungen treffen

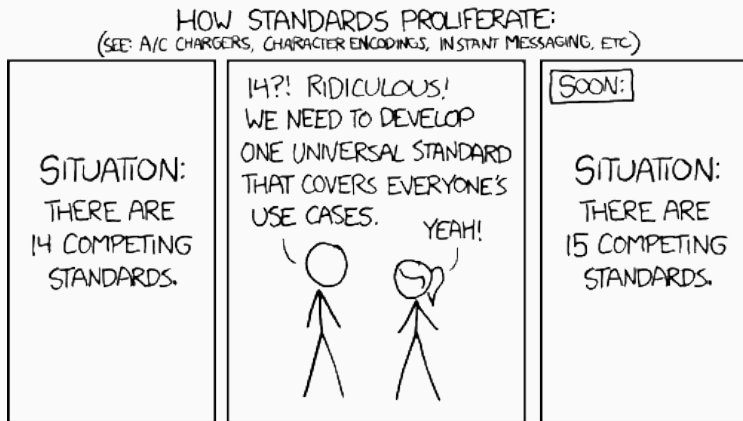
- Testbarkeit
 - ▶ Automatisierbarkeit
- Wartbarkeit
- Erweiterbarkeit
- Kompatibilität

⇒ Standards setzen / Entscheidungen treffen

Welche Ziele soll die Software nun erfüllen?

- Testbarkeit
 - Automatisierbarkeit
 - Wie gesagt Fehler teuer
 - Wartbarkeit und Erweiterbarkeit
 - Dinge im Code ändern sollte einfach sein
 - Andere Menschen sollen Projekt weiterentwickeln können
 - Kompatibilität
 - Windows/MacOs/Linux/Web
- ⇒ wie machen wir das? **Standards setzen** / Entscheidungen treffen

Überleitung xkcd





<https://xkcd.com/927/>

- Also: Keine Standards selber setzen wenn es bereits Standards gibt!
- Diese „Standards“ finden wir als Design-Patterns
- Um uns eine **Meinung über Designpattern** (Standardlösungen) zu bilden müssen wir uns **Paradigmen** (Programmierstile ansehen)

Unterteilung:

- **Imperative**-Programmierung
 - ▶ **Objekt-Orientierte**-Programmierung (OOP)
- **Deklarative**-Programmierung

Unterscheidung: **State-Frage!**

IAWAKEE – Lach und Sachgeschichten aus Mikroklimaforschung und Informatik

└ Einführung Coding-Paradigmen

└ Paradigmen – Einführung

Unterteilung:

- Imperative-Programmierung
 - Objekt-Orientierte-Programmierung (OOP)
- Deklarative-Programmierung

Unterscheidung: **State-Frage!**

Unterteilung:

- **Imperative**-Programmierung (Kennt ihr schon)
 - **Objekt-Orientierte**-Programmierung (OOP) (Kennt ihr auch schon)
- **Deklarative**-Programmierung

Unterscheidung: **State-Frage!**

Was ist State?

- Anwendungszustand
- Die Gesamtheit des Zustands in dem sich ein Programm befindet

State Frage: Wie wird **state** in einer Anwendung gehandhabt?

IAWAKEE – Lach und Sachgeschichten aus Mikroklimaforschung und Informatik

Einführung Coding-Paradigmen

Paradigmen – State-Frage

Was ist State?

- Anwendungszustand
- Die Gesamtheit des Zustands in dem sich ein Programm befindet

State Frage: Wie wird **state** in einer Anwendung gehandhabt?

Was ist State?

- Anwendungszustand
- Die Gesamtheit des Zustands in dem sich ein Programm befindet

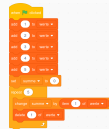
State Frage: Wie wird **state** in einer Anwendung gehandhabt?

Paradigmen – Imperative Programmierung

```
werte = [1,2,3,4,5]
summe = 0
for wert in werte:
    summe = summe + wert
```



```
werte = [1,2,3,4,5]
summe = 0
for wert in werte:
    summe = summe + wert
```



- Beispiel
- Auch gutes Beispiel für die Mächtigkeit von Python vs Scratch

Imperative Programmierung

- Anwendungszustand muss bei jeder Anweisung berücksichtigt werden
- Beschreibt wird **wie** etwas durchgeführt wird

Imperative Programmierung

- Anwendungszustand muss bei jeder Anweisung berücksichtigt werden
- Beschreibt wird **wie** etwas durchgeführt wird

- Kennt ihr schon
- Reguläre Programmierung, alles was für euch halbwegs intuitiv ist
- Keine Klassen
- Untertypen
 - z.b. methodische Programmierung (funktionen)
- Anwendungszustand muss bei jeder Anweisung berücksichtigt werden
 - state ist extern zum code
- Beschreibt wird **wie** etwas durchgeführt wird
 - Wir geben den Computer eine Anleitung
 - klingt so als gäbe es keine alternative, gibt es aber

Paradigmen – Objekt-Orientierte Programmierung

```
class Person
    def __init__(self, name):
        self.name = name
    def greet(self):
        print(self.name + " says hello!")
```

```
ada = Person("Ada Lovelace")
ada.greet()
```

```
alan = Person("Alan Turing")
alan.greet()
```

```
class Person
    def __init__(self, name):
        self.name = name
    def greet(self):
        print(self.name + " says hello!")

ada = Person("Ada Lovelace")
ada.greet()

alan = Person("Alan Turing")
alan.greet()
```

- Bsp

Objekt-Orientierte Programmierung

- Anwendungszustand wird in Klassen gekapselt (Datenkapselung)
 - ▶ Kein globaler¹ State; State ist lokal²
- Beschreibt **wie** etwas durchgeführt wird

¹global = von überall sichtbar/veränderbar

²lokal = nur lokal sichtbar/veränderbar

Objekt-Orientierte Programmierung

- Anwendungszustand wird in Klassen gekapselt (Datenkapselung)
 - Kein globaler¹ State; State ist lokal²
- Beschreibt wird **wie** etwas durchgeführt wird

¹ global := von überall sichtbar/veränderbar
² lokal := nur lokal sichtbar/veränderbar

Objekt-Orientierte Programmierung

- Anwendungszustand wird in Klassen gekapselt (Datenkapselung)
 - Kein globaler State!
 - d.h. State sollte nicht unnötig außerhalb derselben Klasse verfügbar sein.
 - > Globaler state: Überall sichtbar/veränderbar
 - > lokaler state: Nur lokal (z.B. in der selben Klasse) sichtbar
- Beschreibt wird **wie** etwas durchgeführt wird

Paradigmen – Deklarative Programmierung

Deklarative Programmierung

Zielsetzung:

- (Veränderbarer) Anwendungszustand existiert nicht.
 - ▶ Setzen wir einmal eine variable dürfen wir sie nicht verändern
 - ▶ State wird mit hilfe von Funktionen „gekapselt“, state darf nicht außerhalb von Funktionen erscheinen

```
a = 1
def main():
    global a
    print(a+2)
```

```
def main():
    a = 1
    b = 2
    a = a + b
    print(a)
```

```
import time

def timestamp_add_one():
    ts = time.time()
    return ts + 1
```

- (Veränderbarer) Anwendungszustand existiert nicht.

► Setzen wir einmal eine variable dürfen wir sie nicht verändern

► State wird mit Hilfe von Funktionen „gekapselt“, state darf nicht außerhalb von Funktionen erscheinen

```

a = 1          def main():          import time
def main():    a = 1                def timestamp_add_one():
    print(a*2)  b = 2                def timestamp_add_one():
                                    return ts + 1
                                print(a)

```

Paradigmen Deklarative Programmierung

Deklarative Programmierung

Zielsetzung:

- (Veränderlicher) Anwendungszustand existiert nicht.
 - Setzen wir einmal eine variable dürfen wir sie nicht verändern
 - State wird in Funktionen „gekapselt“, state darf nicht außerhalb von Funktionen erscheinen

```

a = 100
def main():
    a = 1
    b = 2
    print(a+b)

```

```

def main():
    a = 1
    b = 2
    c = a + b
    print(c)

```

```

def add(a,b):
    return a + b

```

```

def main():
    a = 1
    b = 2
    print(add(a,b))

```

Ein naiver Versuch:

```
def main():  
    werte = [1,2,3,4,5]  
    summe = 0  
    for wert in werte:  
        summe = summe + wert  
  
main()
```

Paradigmen – Deklarative Programmierung

Ein naiver Versuch:

```
def main():  
    werte = [1,2,3,4,5]  
    summe = 0  
    for wert in werte:  
        summe = summe + wert
```

main()

```
def main():  
    werte = [1,2,3,4,5]  
    summe = 0  
    for wert in werte:  
        summe = summe + wert
```

main()

Paradigmen – Deklarative Programmierung

Neue Spielregel:

```
def f(b):  
    return b() + 2  
  
def a():  
    return 1  
  
def main():  
    funktion = a  
    print(funktion()) # schreibt 1  
    print(f(funktion)) # schreibt 3  
    print(f(a))       # schreibt 3
```

Neue Spielregel:

```
def f(b):  
    return b() + 2
```

```
def a():  
    return 1
```

```
def main():  
    funktion = a  
    print(funktion())    # schreibt 1  
    print(f(funktion))  # schreibt 3  
    print(f(a))         # schreibt 3
```

Ein Designpattern!

Das „Callback-Pattern“:

Eine Funktion kann als Parameter einer anderen Funktion verwendet werden.

Paradigmen – Deklarative Programmierung

Neue Spielregel:

```
while True:  
    print("to infinity and beyond!")
```



```
def do_forever():  
    print("to infinity and beyond!")  
    do_forever()
```


Paradigmen – Deklarative Programmierung

Neue Spielregel:

```
while True:  
    print("to infinity and beyond!")
```



```
def do_forever():  
    print("to infinity and beyond!")  
    do_forever()
```

Kein Designpattern!

Aber ein Konzept – „Rekursion“:

Eine Funktion kann sich selbst aufrufen.

Paradigmen – Deklarative Programmierung

```
def foreach_once(fn, liste, index):  
    fn(liste[index])  
    if index < (len(liste)-1):  
        foreach_once(fn, liste, index + 1)
```

```
def foreach(fn, liste):  
    foreach_once(fn, liste, 0)
```

```
def do_for_all(element):  
    print(element)
```

```
def main():  
    liste = [1,2,3,4,5]  
    foreach(do_for_all, liste)
```

Paradigmen – Deklarative Programmierung

```
def fold_once(fn, akk, liste, index):  
    if index < len(liste):  
        newakk = fn(akk, liste[index])  
        res = fold_once(fn, newakk, liste, index + 1)  
        return res  
    else:  
        return akk  
  
def fold(fn, akk, liste):  
    return fold_once(fn, akk, liste, 0)  
  
def do_for_all(akk, wert):  
    return akk + wert  
  
def main():  
    liste = [1,2,3,4,5]  
    akk = 0  
    summe = fold(do_for_all, akk, liste)  
    print(summe)
```

Klingt unnötig kompliziert?

- Ein einfacheres Beispiel:

```
werte = [1,2,3,4,5]
summe = sum(werte)
print(summe)
```

- Kann wie folgt implementiert werden:

```
def sum(werte):
    def do_for_all(akk, wert):
        return akk + wert

    werte = [1,2,3,4,5]
    akk = 0
    summe = fold(do_for_all, akk, werte)
    return summe
```

Paradigmen – Deklarative Programmierung

```
def main(werte):  
    summe = 0  
    for wert in werte:  
        summe = summe + wert  
    return summe
```

```
def main(werte):  
    return sum(werte)
```

IAWAKEE – Lach und Sachgeschichten aus Mikroklimaforschung und Informatik

└ Einführung Coding-Paradigmen

└ Deklarative Programmierung

└ Paradigmen – Deklarative Programmierung

```
def main(werte):  
    summe = 0  
    for wert in wert:  
        summe = summe + wert  
    return summe
```

```
def main(werte):  
    return sum(werte)
```

- Links: Imperative Programmierung
 - Gehe werte in der Liste Durch
 - > Rechne den aktuellen Listenwert auf die Summe hinzu
 - Gebe Summe zurück
- Rechts: Deklarative Programmierung
 - Das Ergebnis ist die Summe aller Werte.
 - Beschreiben des Ergebnisses statt

Paradigmen – Deklarative Programmierung

```
let werte = [0.2912, 0.421, ...];  
let res = werte.iter()  
    .map(|wert| wert * 100)  
    .map(|wert| wert.round())  
    .filter(|i| i > 50)  
    .fold(0, |avg, i| avg + i / werte.len());
```

Paradigmen – Deklarative Programmierung

```
let werte = [0.2912, 0.421, ...];  
let res = werte.iter()  
    .map(|wert| wert * 100)  
    .map(|wert| wert.round())  
    .filter(|i| i > 50)  
    .fold(0, |avg, i| avg + i / werte.len());
```

```
SELECT AVG(gerundeter_wert) AS durchschnitt_gefiltert  
FROM (  
    SELECT ROUND(wert * 100.0) AS gerundeter_wert  
    FROM werte_tbl  
    ) t WHERE gerundeter_wert > 50;
```



```
let wert = [0.2912, 0.421, ...];  
let res = wert.iter()  
    .map(|wert| wert * 100)  
    .map(|wert| wert.round())  
    .filter(|i| i > 50)  
    .fold(0, |avg, i| avg + i / wert.len());  
  
SELECT AVG(gerundeter_wert) AS durchschnitt_gefiltert  
FROM (  
    SELECT ROUND(wert * 100.0) AS gerundeter_wert  
    FROM werte_tbl  
) t WHERE gerundeter_wert > 50;
```

1. Das ergebnis ist die Repräsentation aller Werte als Prozent-, statt Kommazahlen
2. Diese Prozentzahlen sind gerundet
3. Nur Prozentzahlen > 50 werden ausgewählt
4. Von den ausgewählten Zahlen wird der Durchschnitt errechnet.
5. SQL ebenfalls deklarativ!

Paradigmen – Deklarative Programmierung

```
werte = [0.2912, 0.421, ...]
```

```
def to_percent(wert):  
    return wert * 100  
def rounded(wert):  
    return round(wert)  
def is_gt_50(wert):  
    return wert > 50  
def build_avg(length):  
    def avg(akk, wert):  
        return akk + wert / length  
    return avg  
res = werte .map(to_percent)  
            .map(rounded)  
            .filter(is_gt_50)  
            .fold(0, build_avg(len(werte)));
```

```
werte = [0.2912, 0.421, ...]
```

```
avg = 0  
for wert in werte:  
    a = wert * 100  
    b = round(a)  
    if b > 50:  
        c = b  
        avg = avg + c / len(werte)
```

```
werte = [0.2012, 0.421, ...]

def to_percent(wert):
    return wert * 100
def rounded(wert):
    return round(wert)
def is_gt_50(wert):
    return wert > 50
def build_avg(length):
    def avg(acc, wert):
        return acc + wert / length
    return avg

res = werte.map(to_percent)
               .map(rounded)
               .filter(is_gt_50)
               .fold(0, build_avg(len(werte)))

werte = [0.2012, 0.421, ...]

avg = 0
for wert in werte:
    a = wert * 100
    b = round(a)
    if b > 50:
        c = b
    avg = avg + c / len(werte)
```

- etwas Gefährlich
- IdR nicht wirklich optimiert, aber möglich!
- Rust z.B. kann eigentlich rekursive Prozesse → iterativ umsetzen
- Rekursive Prozesse lassen sich problemlos iterativ darstellen

Paradigmen – Deklarative Programmierung

Weiteres Designpattern

Eine Funktion, die mehrere Parameter nimmt kann wie folgt in eine Serie von Funktionen aufgebrochen werden:

```
def add(a, b):  
    return a + b
```

```
def add_curried(a):  
    def add_b(b):  
        return add(a,b)  
    return add_b
```

```
add5 = add_curried(5) # Erzeugt eine Funktion, die 5 + b rechnet  
print(add5(3))         # Ausgabe: 3 + 5 = 8
```

Diese Praxis heißt **Currying**.

Pattern in der Objekt-Orientierten Programmierung

Pattern in der Objekt-Orientierten Programmierung

Factory Method Pattern

```
public interface Postage { String type(); }

public class RegularPostage implements Postage {
    public String type() { return "Regular"; }
}

public class ExpressPostage implements Postage {
    public String type() { return "Express"; }
}

public final class PostageFactory {
    private PostageFactory() {}
    public static Postage by_price(double price_for_product) {
        if (price_for_product > 30) {
            return new ExpressPostage();
        } else {
            return new RegularPostage();
        }
    }
}
```

└ Pattern in der Objekt-Orientierten Programmierung

Factory Method Pattern

```
public interface Postage { String type(); }

public class RegularPostage implements Postage {
    public String type() { return "Regular"; }
}

public class ExpressPostage implements Postage {
    public String type() { return "Express"; }
}

public final class PostageFactory {
    private PostageFactory() {}
    public static Postage by_price(double price_for_product) {
        if (price_for_product > 30) {
            return new ExpressPostage();
        } else {
            return new RegularPostage();
        }
    }
}
```

- Wenn verschiedene Klassen ein Interface implementieren können wir zur Laufzeit entscheiden ein Objekt welcher konkreten Klasse wir zurückgeben
- Methode by_price von Klasse PostageFactory gibt interface Postage zurück.
 - Methode heißt Factory method / „fabrik methode“

- Wenn verschiedene Klassen ein Interface implementieren können wir zur Laufzeit entscheiden ein Objekt welcher konkreten Klasse wir zurückgeben
 - ▶ Hier gibt die Methode `by_price` der Klasse `PostageFactory` das interface `Postage` zurück.
 - ▶ Diese Methode heißt Factory method oder „Fabrik Methode“

Pattern in der Objekt-Orientierten Programmierung

Builder Pattern

```
class House:
    def __init__(self, walls, roof, door):
        self.walls = walls
        self.roof = roof
        self.door = door

class HouseBuilder:
    def __init__(self):
        self._walls = "concrete" # Houses have concrete walls by default
        self._roof = "tile"      # Houses have tile roofs by default
        self._door = "glass"     # Houses have glass by default
    def walls(self, style):
        self._walls = style
        return self
    def roof(self, style):
        self._roof = style
        return self
    def door(self, style):
        self._door = style
        return self
    def build(self):
        return House(self._walls, self._roof, self._door)
```

└ Pattern in der Objekt-Orientierten Programmierung

Builder Pattern

```

class House:
    def __init__(self, walls, roof, door):
        self.walls = walls
        self.roof = roof
        self.door = door

class HouseBuilder:
    def __init__(self):
        self._walls = "concrete" # House has concrete walls by default
        self._roof = "tile" # House has tile roof by default
        self._door = "glass" # House has glass by default
    def walls(self, style):
        self._walls = style
        return self
    def roof(self, style):
        self._roof = style
        return self
    def door(self, style):
        self._door = style
        return self
    def build(self):
        return House(self._walls, self._roof, self._door)

```

- Hier Übertragen wir die Verantwortung die Klasse House zu konstruieren an die Klasse HouseBuilder
- Dadurch müssen wir den Konstruktor von House nicht mehr selber aufrufen und können dadurch Standard Werte festlegen.
- Unnötig in python in diesem kontext

Pattern in der Objekt-Orientierten Programmierung

- Hier Übertragen wir die Verantwortung die Klasse House zu konstruieren an die Klasse HouseBuilder
- Dadurch müssen wir den Konstruktor von House nicht mehr selber aufrufen und können dadurch Standard Werte festlegen.

```
house = HouseBuilder()  
        .walls("brick")  
        .roof("flat")  
        .door("oak")  
        .build()
```

Zusammenfassung

Ein Kompromiss:

- Objektorientierte Programmierung
 - ▶ State sollte in Klassen gekapselt sein
 - ▶ Globalen State vermeiden
- Deklarative Programmierung
 - ▶ State allgemein vermeiden
 - ▶ State-Änderungen vermeiden
 - ▶ Deklarative-Primitives nutzen
 - ▶ Immutability > Datenkapselung
- Design Pattern
 - ▶ Bieten Struktur, Standardisierung und Namenskonvention

Ein Kompromiss:

- Objektorientierte Programmierung
 - ▶ State sollte in Klassen gekapselt sein
 - ▶ Globalen State vermeiden
- Deklarative Programmierung
 - ▶ State allgemein vermeiden
 - ▶ State-Änderungen vermeiden
 - ▶ Deklarative-Primitives nutzen
 - ▶ Immutability > Datenkapselung
- Design Pattern
 - ▶ Bieten Struktur, Standardisierung und Namenskonvention

- Am Ende kommt es auf die State Frage an
- Objektorientierte Programmierung
 - State sollte in Klassen gekapselt sein
 - Globaler State ist zu vermeiden
- Deklarative Programmierung
 - State-Änderungen / Änderbarer State ist zu vermeiden
 - Deklarative-Primitives nutzen
 - > Funktionen wie reduce, forEach, etc. werden von Sprache idR. bereits bereitgestellt
 - Zu starke Gewichtung dieser Ideale in den meisten Kontexten schwierig, etwas Deklarative Programmierung macht Code häufig besser
 - Immutability > Datenkapselung
- Design Pattern

Vielen Dank für Eure Aufmerksamkeit!

Diese Präsentation:

https://github.com/chaosFaktor/iawakee_for_dummies/tree/master/output



Deklarative Programmierung

„Dear Functional Bros” – CodeAesthetic <https://www.youtube.com/watch?v=nuML9SmdbJ4>

Software Design

CodeAesthetic (Kanal) <https://www.youtube.com/@CodeAesthetic>

Software Design

„Gang of Four Design Patterns” <https://www.scribd.com/doc/135313382/Gang-of-Four-Design-Patterns-4-0-pdf>

Einfachere Erklärung: <https://refactoring.guru/>

Informatik Allgemein

The Coding Train <https://www.youtube.com/playlist?list=PLRqwX-V7Uu6Zy51Q-x9tMWIv9cue0FTFA>

CCC Hackerethik <https://www.ccc.de/de/hackerethik>

Wargames

„CTF: WTF?“ <https://media.ccc.de/v/38c3-ctf-wtf-capture-the-flag-fr-ein>

Over the Wire <https://overthewire.org/wargames/>

Pixelflut <https://c3pixelflut.de/>

Anderes

Kinder, es tut mir unendlich leid ...

[https://media.ccc.de/v/
gpn21-29-kinder-es-tut-mir-undendlich-leid](https://media.ccc.de/v/gpn21-29-kinder-es-tut-mir-undendlich-leid)

„Der Thüring-Test für Wahlsoftware“ – Linus Neumann und Thorsten Schröder

[https://media.ccc.de/v/
38c3-der-thring-test-fr-wahlsoftware](https://media.ccc.de/v/38c3-der-thring-test-fr-wahlsoftware)

„Wir wissen wo dein Auto steht“ – Kreil und Flüpke

[https://media.ccc.de/v/
38c3-wir-wissen-wo-dein-auto-steht-volksda](https://media.ccc.de/v/38c3-wir-wissen-wo-dein-auto-steht-volksda)

Anderes

„How to build a submarine and survive” – Nico und Elias

https://media.ccc.de/v/37c3-11828-how_to_build_a_submarine_and_survive

Simone Giertz

<https://www.youtube.com/@simonegiertz>

Anderes

Lilith Wittmann

<https://links.lilithwittmann.de/> bzw
<https://media.ccc.de/search?p=Lilith+Wittmann>

„Jens Spahns credit score is 'very good'” – Lilith Wittmann

https://media.ccc.de/v/camp2023-57571-jens_spahns_credit_score_is_very_good

Anderes

„Heimlich Manöver“ – Arne Sems-
rott

[https://media.ccc.de/v/
37c3-11689-heimlich-manover](https://media.ccc.de/v/37c3-11689-heimlich-manover)

Fnord - Nachrichtenrückblick – Fefe und Atoth

[https://media.ccc.de/v/
38c3-fnord-nachrichtenrckblick-202](https://media.ccc.de/v/38c3-fnord-nachrichtenrckblick-202)

Fefe

<https://blog.fefe.de>

Anderes

„Illegal Infrastructure: 12 years of hosting in the greyzone” – Kolja und Mark

[https://media.ccc.de/v/](https://media.ccc.de/v/38c3-illegal-infrastructure-12-years-of-hos)

[38c3-illegal-infrastructure-12-years-of-hos](https://media.ccc.de/v/38c3-illegal-infrastructure-12-years-of-hos)

Attack Mining: How to use distributed sensors to identify and take down adversaries (Fortgeschritten)

[https://media.ccc.de/v/](https://media.ccc.de/v/38c3-attack-mining-how-to-use-distributed-s)

[38c3-attack-mining-how-to-use-distributed-s](https://media.ccc.de/v/38c3-attack-mining-how-to-use-distributed-s)

„Alles ist 1 außer der 0” (Film)